

sql笔记

2.库的操作

2.1字符集和校验集

查看字符集：

```
show charset
```

查看检验集：

```
show collation
```

查看默认的字符集：

```
show variables like 'character_set_database';
```

查看默认的校验集：

```
show variables like 'collation_database';
```

2.2创建和删除

创建数据库：

```
create database d1 character set utf8mb4 collate utf8mb4_general_ci;
```

数据库最重要的就是字符集和校验集。

UTF-8 Multi-Byte 4-Byte General Case-Insensitive Collation

Unicode Transformation Format - 8-bit

删除数据库：

```
drop database d1;
```

2.3数据库的查看

查看数据库的个数：

```
show databases;
```

查看如何创建数据库：

```
show create database d1;
```

使用数据库：

```
use d1;
```

查看链接数

```
show processlist;
```

3.表的操作

3.1创建表和删除表：

创建表：

```
create table users2 (  
id int comment '序号',  
name varchar(20) comment '名称',  
passwd char(32) comment '密码',  
birthday date comment '生日' ) character set utf8mb4 collate utf8mb4_general_ci  
engine InnoDB;
```

表的：字符集，校验集，存储引擎。

删除表：

```
drop table users2;
```

3.2查看表：

表的逻辑结构：

```
desc users2;
```

表的创建语句：

```
show create table users2 \G
```

3.3表的性质修改：

修改表的名称：

```
alter table users2 rename to user
```

增加一列属性：

```
alter table user add image_path varchar(30) comment '用户头像';
```

删除一列属性：

```
alter table user drop id;
```

修改属性：

```
alter table user modify name int;
```

```
alter table user change name xingming varchar(30);
```

总结：

```
alter table tb_name
rename to    // 新名称
add          // 添加新的一行
drop         // 删除一行
modify       // 修改属性，列名称不变
change       // 修改名字和属性
```

查看表的操作

```
desc tb_name;
show create table tb_name \G
show tb_name;
```

4.数据类型

4.1整型

类型	存储空间	有符号范围	无符号范围
TINYINT	1 字节	-128 ~ 127	0 ~ 255
SMALLINT	2 字节	-32768 ~ 32767	0 ~ 65535
MEDIUMINT	3 字节	-8388608 ~ 8388607	0 ~ 16777215
INT	4 字节	-2147483648 ~ 2147483647	0 ~ 4294967295
BIGINT	8 字节	$-2^{63} \sim 2^{63}-1$	$0 \sim 2^{64}-1$

默认是有符号的。可以指定为无符号的。

mysql：中数据类型也是一种约束的。

mysql中：变量名：类型

```
create table t11(  
    t1 tinyint,  
    t2 tinyint unsigned);
```

bit[(M)]: 位字段类型。M表示每个值的位数，范围从1到64。如果M被忽略，默认为1。

4.2浮点型

类型	占用空间	数值性质	大约有效精度
<code>FLOAT</code>	4 字节	近似值	约 6 ~ 7 位十进制有效数字
<code>DOUBLE</code>	8 字节	近似值	约 15 ~ 16 位十进制有效数字
<code>DECIMAL(M,D)</code>	根据 M、D 决定	精确值	由 M 决定，最多保存 M 位数字

注意:浮点型会四舍五入的。

四舍五入也不能超过范围的。

4.3字符型

char和**varchar**类型。

对比项	<code>CHAR(M)</code>	<code>VARCHAR(M)</code>
长度性质	固定长度	可变长度
M 的含义	最多存储 M 个字符	最多存储 M 个字符
长度范围	0 ~ 255 个字符	理论上 0 ~ 65535 个字符
存储空间	按声明长度处理，不足时补空格	按实际内容长度存储
额外长度信息	通常不需要记录实际长度	额外使用 1 或 2 字节记录长度
尾部空格	存储时补空格，查询时通常删除	不自动补空格，尾部空格通常保留
适合的数据	长度基本固定	长度变化较大

char存储的是字符：就是我们实际看到的字符样子。0-255之间的

varchar存储的底层是：65535个字节，所以存储的字符个数还需要和编码信息统一，而且底层还有存储的容量计数的。

4.4日期类型

类型	标准格式	占用空间	主要范围	常见用途
YEAR	YYYY	1 字节	1901 ~ 2155, 以及 0000	出生年份、生产年份
DATE	YYYY-MM-DD	3 字节	1000-01-01 ~ 9999-12-31	生日、入职日期、订单日期
TIME	HH:MM:SS	3 字节	-838:59:59 ~ 838:59:59	时刻、持续时间、时间间隔
DATETIME	YYYY-MM-DD HH:MM:SS	5 字节	1000-01-01 ~ 9999-12-31	业务日期时间、预约时间
TIMESTAMP	YYYY-MM-DD HH:MM:SS	4 字节	1970-01-01 ~ 2038-01-19	创建时间、更新时间、日志时间

date, datetime。需要程序员自己插入，插入需要安装字符串的格式进程插入的。

timestamp。时间戳，每一次操作数据库，会形成一个时间戳的。

4.5枚举和集合

enum('选项1','选项2','选项3',...);

insert的时候，可以直接插入1, 2, 3就是对应的选项了。枚举只能选择一个的。

set('选项值1','选项值2','选项值3', ...);

insert的时候，可以直接插入数字， $0-2^n-1$ 次方的数字。 注意从左往右的数字。

mysql的字段进行比较的时候，都是严格的匹配的

```
mysql> desc t10;
```

Field	Type	Null	Key	Default	Extra
name	varchar(30)	YES		NULL	
gender	enum('男','女')	YES		NULL	
hobby	set('登山','羽毛球','乒乓球','足球','游泳')	YES		NULL	

```
3 rows in set (0.00 sec)

mysql> select * from t10 where gender=2;
```

name	gender	hobby
lic2号	女	登山
lic2号	女	羽毛球
曹操	女	登山,羽毛球,游泳
曹操	女	登山
曹操	女	羽毛球
曹操	女	登山,羽毛球
曹操	女	乒乓球
曹操	女	登山,羽毛球,乒乓球
曹操	女	游泳
曹操	女	登山,羽毛球,乒乓球,足球,游泳

```
10 rows in set (0.00 sec)

mysql>
```

直接用变量名=数值进行查询
这是严格匹配的

find_in_set函数的使用。

```
mysql>
mysql> select * from t10;
```

name	gender	hobby
lic1号	男	登山
lic2号	男	登山
lic2号	女	登山
lic2号	男	登山
lic2号	女	羽毛球
操作	男	登山,羽毛球
操作	男	登山,羽毛球,游泳
曹操	女	登山,羽毛球,游泳
曹操	女	登山
曹操	女	羽毛球
曹操	女	登山,羽毛球
曹操	女	乒乓球
曹操	女	登山,羽毛球,乒乓球
曹操	女	游泳
曹操	女	登山,羽毛球,乒乓球,足球,游泳

```
15 rows in set (0.00 sec)

mysql>
mysql>
mysql>
mysql> select * from t10 where find_in_set('登山', hobby);
```

name	gender	hobby
lic1号	男	登山
lic2号	男	登山
lic2号	女	登山
lic2号	男	登山
操作	男	登山,羽毛球
操作	男	登山,羽毛球,游泳
曹操	女	登山,羽毛球,游泳
曹操	女	登山
曹操	女	登山,羽毛球
曹操	女	登山,羽毛球,乒乓球
曹操	女	登山,羽毛球,乒乓球,足球,游泳

```
11 rows in set (0.00 sec)

mysql>
```

5约束

5.1 非空约束

null表示什么都没有的，null不参与计算的。注意null和空白字符的区别的。

not null 非空的。

注意：数据库默认字段基本都是字段为空，但是实际开发时，尽可能保证字段不为空，因为数据为空没办法参与运算。

5.2 默认值

default: 设置了，用户的数据存在了，就用用户的，不存在就用自己的。

default 和 not null不冲突的。

mysql默认是null的。这是mysql的优化的。

5.3 描述列。

相当于语言的注释。comment '注释的信息'。

列描述：comment，没有实际含义，专门用来描述字段，会根据表创建语句保存，用来给程序员或DBA 来进行了解。

5.4 zerofill

零填充约束。

显示方面的约束

`ZEROFILL` 是 MySQL 早期用于数字格式化显示的修饰符，它会根据显示宽度在数字左侧补零，并自动添加 `UNSIGNED` 属性。

它不会改变数据的实际存储值和存储空间，只影响查询显示效果。

从 MySQL 8.0.17 开始已被官方弃用，实际开发中通常使用 `LPAD()` 或由应用程序完成补零格式化。

`hex()` 转换成16进制的。

5.5 主键约束

主键：primary key用来唯一的约束该字段里面的数据，不能重复，不能为空，一张表中最多只能有一个

主键；主键所在的列通常是整数类型。

主键约束默认就是 非空的

```
update t17 set name = '刘备' where id = 2; // 主键存在方便更新。
alter table t17 drop primary key;          // 因为主键一张表一个，不要告诉我哪一个列的。
alter table t17 add primary key(id);        // 注意建表之前就使用 主键。
delete from t17 where name = '刘备';
```

复合主键

```
mysql> create table t18(
    -> id int unsigned,
    -> coures char(10) comment '课程',
    -> score tinyint unsigned default 60 comment '成绩',
    -> primary key(id, coures) // 复合主键，最后指定就行了。
    -> );
```

复合主键的语法是：在最后面进行指示。约束的条件只需要有一个不一样就行了的。

5.6自增长

auto_increment：当对应的字段，不给值，会自动的被系统触发，系统会从当前字段中已经有的最大值 +1操作，得到一个新的不同的值。

通常和主键搭配使用，作为逻辑主键。

自增长的特点: 任何一个字段要做自增长，前提是本身是一个索引（key一栏有值）

自增长字段必须是整数

一张表最多只能有一个自增长

设定起始值的

```
create table t20(
    id int primary key auto_increment,
    name varchar(20) not null
)auto_increment = 55;
```

5.7唯一键

对比项	主键约束 <code>PRIMARY KEY</code>	唯一键约束 <code>UNIQUE</code>
主要作用	唯一标识表中的一条记录	保证某列或某组列的数据不重复
是否允许重复	不允许	不允许
是否允许 <code>NULL</code>	不允许	允许 <code>NULL</code>
一张表可以有几个	只能有一个主键	可以有多个唯一键
是否自动具有 <code>NOT NULL</code>	是	否
常见用途	用户编号、订单编号、商品编号	手机号、邮箱、身份证号、用户名
InnoDB 中的特殊作用	默认作为聚簇索引	一般作为唯一二级索引

5.8 外键

外键写自己，引用写父表；先建父表，再建子表；先删子表，再删父表。

```
mysql> desc class;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| id    | int(11)       | NO   | PRI | NULL    |       |
| name  | varchar(32)   | NO   |     | NULL    |       |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql>
mysql>
mysql>
mysql> create table stu(
    -> id int unsigned primary key,
    -> name varchar(20) not null,
    -> telephone varchar(32) unique key,
    -> class_id int ,
    -> foreign key(class_id) references class(id)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
```

FOREIGN KEY	外键
(class_id)	本表的字段
REFERENCES	引用
class(id)	class 表中的 id 字段

6查询

CURD

6.1 insert

```
insert into t22(id, sn, name, qq) values(2, 1, '张三', '2225'); // 全部指定列名
insert into t22 values(2, 1, '张三', '2225'); // 全部不指定列名，默认全插入
insert into t22(id, sn, name, qq) values(12, 126, '曹操', '25'),(14, 13, '孙权', 'wee'); // 多行插入，注意括号
```

查看自增约束的数字

```
show create tb_name \G
```

自增约束可以不指定，会使用默认的下一个值。

更新

```
INSERT INTO 表名(字段1, 字段2, ...)
VALUES(值1, 值2, ...)
ON DUPLICATE KEY UPDATE
    字段1 = 新值,
    字段2 = 新值;
```

没有主键/唯一键冲突：直接 **INSERT**
发生主键/唯一键冲突：**DELETE** 旧记录，再 **INSERT** 新记录

UPDATE：修改原记录
ON DUPLICATE KEY UPDATE：插入失败则修改原记录
REPLACE：插入失败则删除原记录，再重新插入

6.2 select

select

```
select * from exam_result; // 取出整个表的结构
select name, chinese from exam_result; // 取出指定列的数据
select name as 姓名, chinese as 语文, math as 数学, english as 英语 from
exam_result; // 取出指定列，然后指定别名 as可以省略的
select distinct math from exam_result; // math 去重
select distinct math, name from exam_result; // math + name 去重
select distinct math + 2, name from exam_result; // 可以进行计算的取出的数据。
```

```
SELECT * FROM tb_name;
```

- 1.先确定从哪张表读取数据；
- 2.再决定查询结果中显示哪些字段。

where子句

语文成绩在 [80, 90] 分的同学及语文成绩

```
select * from employee where salary >= 10000 and salary <= 12000;
```

select * from employee where salary between 10000 and 12000; // between and 是闭区间的

数学成绩是 58 或者 59 或者 98 或者 99 分的同学及数学成绩

```
select * from employee where age=25 or age=43 or age=35 or age=36;
```

select * from employee where age in(25,43,35,36); // in 可以进行列举的

姓孙的同学 及 孙某同学

```
select * from employee where name like '孙%';
select * from employee where name not like '孙%';
select * from employee where name like '孙_';
select * from employee where name not like '孙_'; // %任意字符 _任意一个字符。 注意区别
```

语文成绩好于英语成绩的同学

```
select name, chinese, english from exam_result where chinese > english;
```

总分在 200 分以下的同学

```
select name, chinese+english+math 总分 from exam_result where chinese + math + english < 200;
```

```
select name, chinese+english+math 总分 from exam_result where chinese + math + english < 200;
```

- | | | |
|----------|---|---|
| 3 | 1 | 2 |
| 1. 先找到表 | | |
| 2. 筛选的条件 | | |
| 3. 展示 | | |

1. FROM: 确定从哪张表中读取数据。
2. WHERE: 根据条件对表中的行进行判断和筛选。
3. SELECT: 选择需要展示的字段, 形成最终查询结果。

null的查询 is null, is not null

order by 排序

安装年龄排序

```
select * from employee order by age asc;  
select * from employee order by age desc;
```

```
// **前面的数据一样的话, 才会按照后面的顺序进行排序.**  
select * from employee order by status asc, salary asc;
```

```
select name, chinese+math+english total from exam_result order by total desc;
```

这里为什么能够使用别名呢? 1.首先拿出 2.然后展示出来 3.然后按照顺序展示的。

1. FROM exam_result
找到数据表
2. WHERE name LIKE '孙%' OR name LIKE '曹%'
筛选姓孙或姓曹的学生
3. SELECT name, math
只展示姓名和数学成绩
4. ORDER BY math DESC
按数学成绩从高到低排序

limit 分页查询

// 第一到第五个

```
select * from exam_result limit 5; // 表开始，连续读取五行
```

// 第二到后面五个

```
select * from exam_result limit 2, 5; // 从二开始，连续读取五行
```

// 注意开始位置，下标默认是从零开始的

```
select * from exam_result limit 3 offset 2; // offset表示起始位置，前面的就是步长
```